Git & GitHub Fundamentals

ASSIGNMENT 1 · SESSION 1 · GROUP D

Full Name	Mira Negra
Student ID	157694
Course Code & Section	3324 / 01
GitHub Username	Amramix108
Report Date	October 15, 2026

Repository: https://github.com/Amramix108/git-session1-Amramix108

1. Repository Setup

I used the supplied STARTER files in `git-session1-Amramix108` and initialized the repository on `main` with `git init -b main`. The required ignore patterns are:

```
local.env
*.log
!important.log
```

I checked the result with `git status --short --ignored`. The environment file and normal logs appear with `!!`; `important.log` appears with `??` and is available to stage. The leading `!` makes an exception to the log pattern [3].

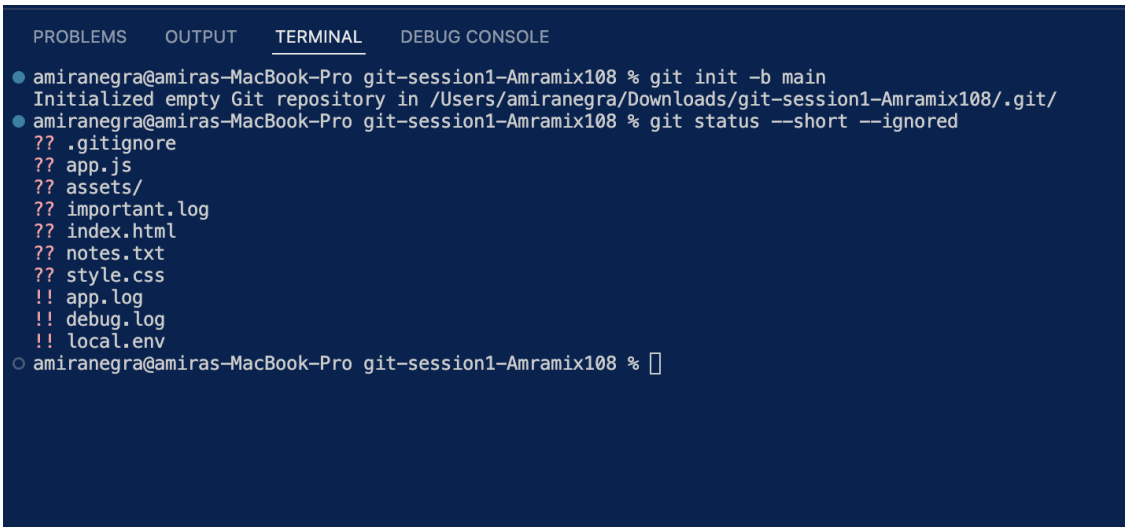


Figure 1. Initialization on main and verification of the required ignore behavior.

The initial commit tracked eight files, including the banner, `important.log`, HTML, CSS, and JavaScript.

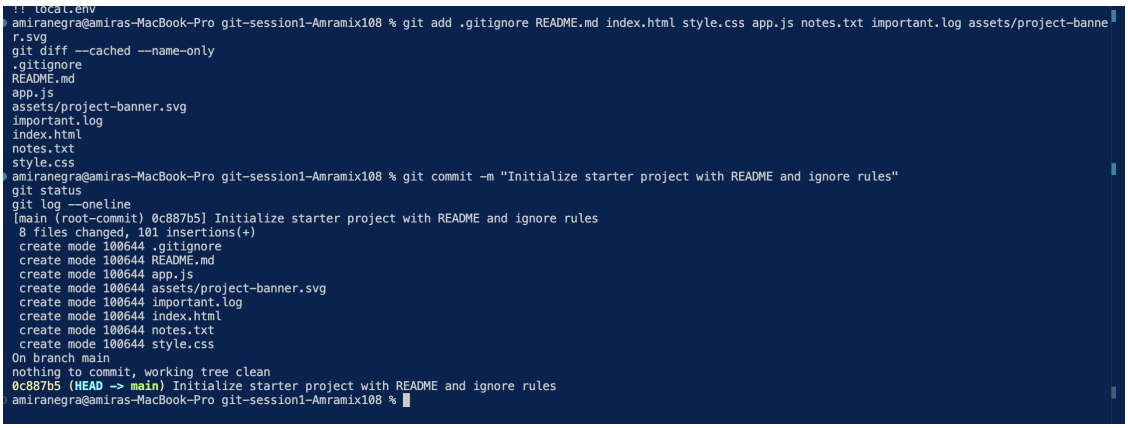


Figure 2. The initial staged file list and successful first commit, 0c887b5.

2. README & Commit History

My README contains my name, a GitHub username field, a project description, technologies, an example command, a profile link, and a reference to the supplied banner. The later README screenshot shows the profile link updated to `Amramix108`. Figure 3 shows the Markdown source.

README.md > # Git & GitHub — Session 1 Assignment > ## Project Image

```
# Git & GitHub — Session 1 Assignment

**Name:** Mira Negra
**GitHub username:** Amaramix108

## Description

A small website used to practice Git repositories, staging, commits,
branches, merging, and synchronization with GitHub.

## Technologies

- HTML
- CSS
- JavaScript
- Git
- GitHub

## Run the project

Open `index.html` in your browser.

## Example command

Check the repository status:

```bash
git status
```

GitHub Profile

[Visit my GitHub profile](https://github.com/redaalch)

Project Image

![Project banner](assets/project-banner.svg)
```

Figure 3. README source containing the required sections and the project-banner reference.

The screenshots document five commits before the report is added. The initial commit covers eight files; later commits separate the stylesheet improvement, About feature, merge, and remote README update:

```
0c887b5  Initialize starter project with README and ignore rules
b3abddc  Add rounded corners to project sections
d684278  Add About section to homepage
5cb5bce  Merge About feature into main
3efa578  Document About feature completion
```

The attempted `Document Git learning goals` commit did not succeed because nothing had been staged. It is therefore absent from this history.

3. Selective Staging

I modified two tracked files: I added Learning Goals to `README.md` and added `border-radius: 8px;` to the section rule in `style.css`. The captured diff includes both files.

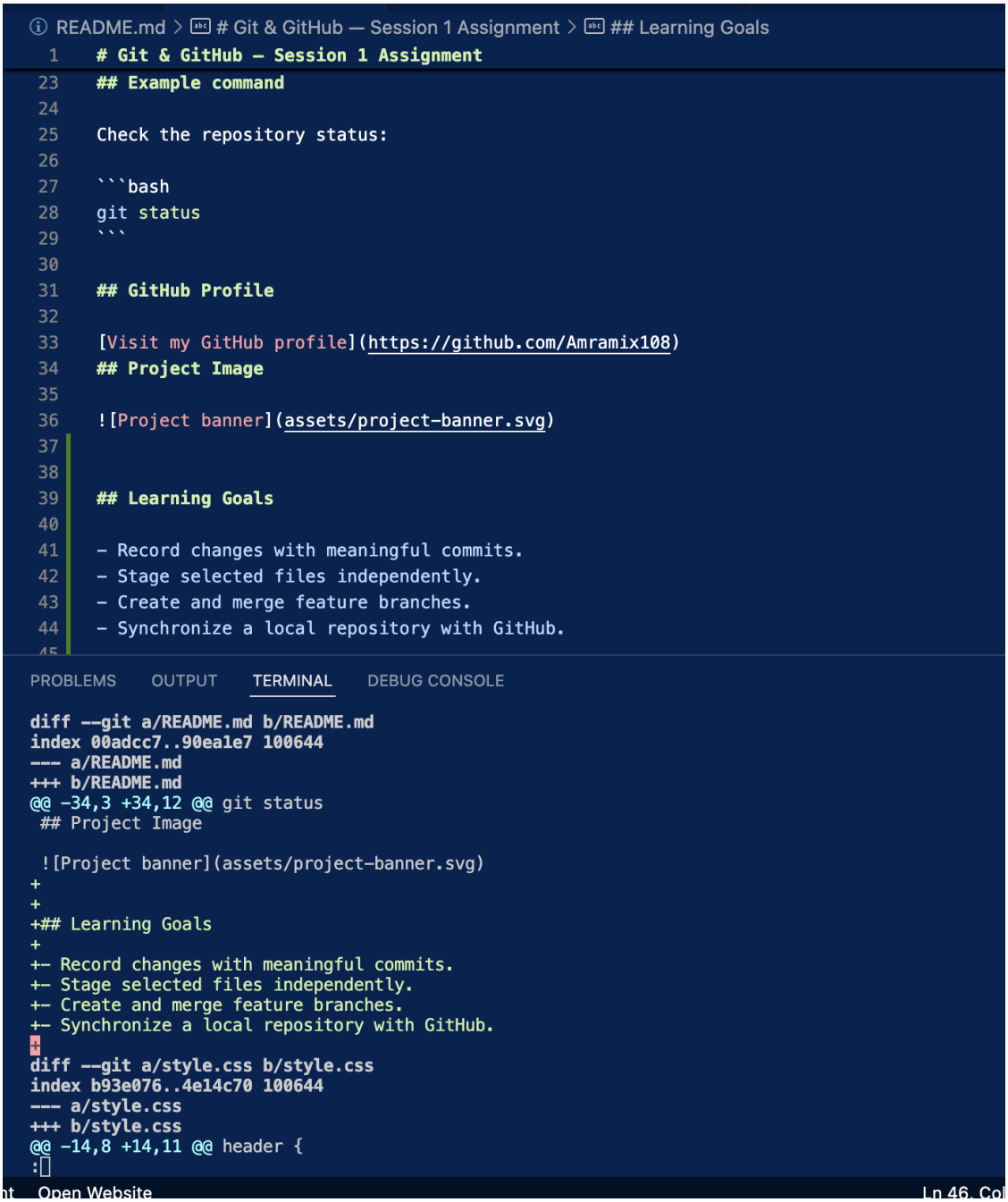


Figure 4. The Learning Goals edit and diff showing both README.md and style.css changes.

`git diff` compares the working files with the staging area. `git diff --cached` compares staged changes with the current commit. These views distinguish edits that are ready to commit from edits still in the working directory.

3. Selective Staging — Commit Result

My first commit attempt reported that no changes had been added to the commit. Both the README and stylesheet were still modified and unstaged.

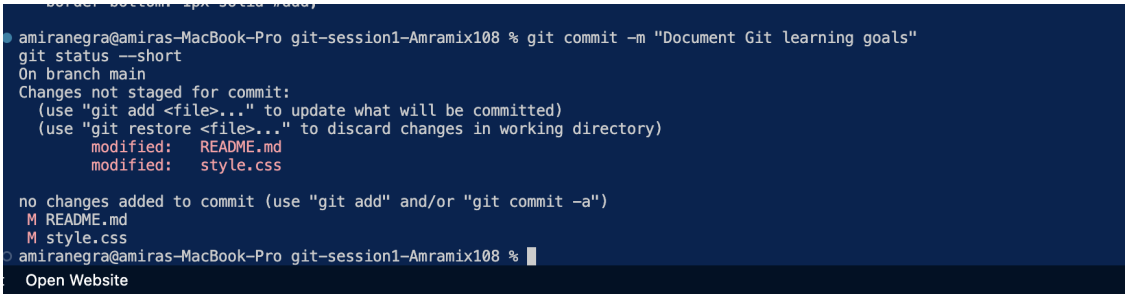


Figure 5. The commit attempt saves nothing because neither changed file has been staged.

I subsequently committed only the stylesheet as `b3abddc`, with the message “Add rounded corners to project sections.” Git reports one changed file, while `README.md` remains modified. This demonstrates selective staging with the CSS change committed first.

```
git commit -m "Add rounded corners to project sections"
git status
git log --oneline
diff --git a/style.css b/style.css
index b93e076..4e14c70 100644
--- a/style.css
+++ b/style.css
@@ -14,8 +14,11 @@ header {
   border-bottom: 1px solid #ddd;
 }
 section {
+  border-radius: 8px;
   background: white;
   margin-top: 20px;
   padding: 20px;
   border: 1px solid #e3e3e3;
 }
+
+
diff --git a/style.css b/style.css
index b93e076..4e14c70 100644
--- a/style.css
+++ b/style.css
@@ -14,8 +14,11 @@ header {
   border-bottom: 1px solid #ddd;
 }
 section {
+  border-radius: 8px;
   background: white;
   margin-top: 20px;
   padding: 20px;
   border: 1px solid #e3e3e3;
 }
+
+
[main b3abddc] Add rounded corners to project sections
 1 file changed, 3 insertions(+)
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
b3abddc (HEAD -> main) Add rounded corners to project sections
0c887b5 Initialize starter project with README and ignore rules
amiranegra@amiras-MacBook-Pro git-session1-Amramix108 %
```

Figure 6. The CSS diff and CSS-only commit; the README remains modified afterward.

4. Branch & Merge

I created `feature/about`, added an About section to `index.html`, and saved it in commit `d684278`. I then returned to `main` and merged the feature using `--no-ff`, creating the explicit merge commit `5cb5bce`.

```
git switch main
git merge --no-ff feature/about -m "Merge About feature into main"
git log --oneline --graph --decorate --all
```

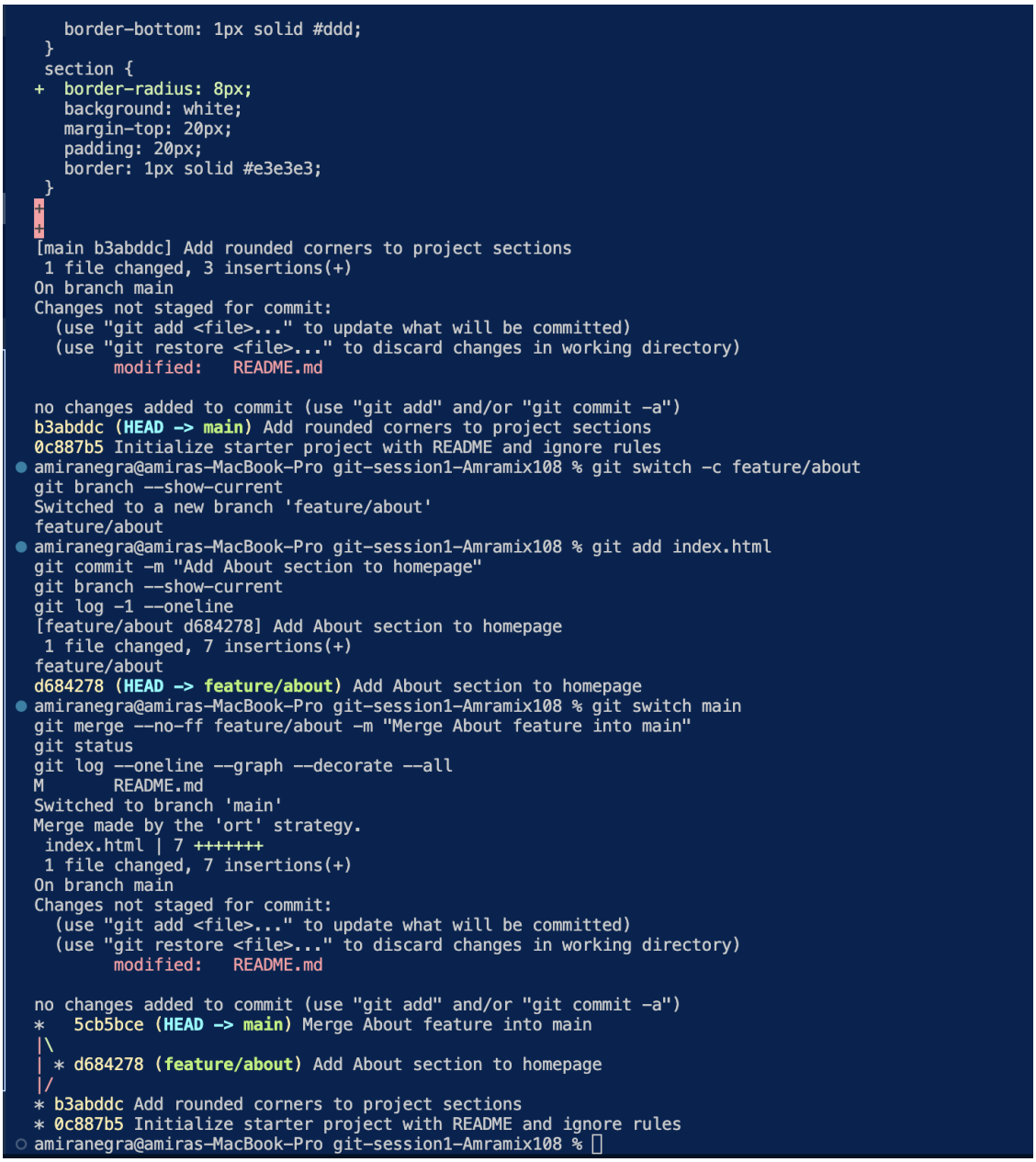


Figure 7. The About feature commit and merge graph. The earlier README edit remains unstaged.

The graph shows the separate feature commit joining main through the merge commit. The About commit changes only the HTML file.

5. Remote Changes

Correcting the remote URL

The first push failed with HTTP 403 because `origin` pointed to another account's repository. Git explicitly reported that permission was denied to `Amramix108`. I corrected the remote URL and checked it with `git remote -v`:

```
git remote set-url origin https://github.com/Amramix108/git-session1-Amramix108.git
git remote -v
```



Figure 8. Terminal excerpt showing the permission error and correction to Mira's own repository.

Publishing main

I then ran `git push -u origin main`. The push succeeded, created the remote main branch, and configured the local branch to track `origin/main`.

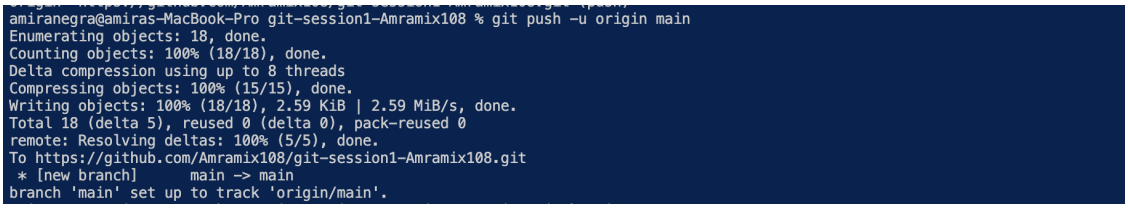


Figure 9. Successful push to Amramix108/git-session1-Amramix108 and upstream tracking confirmation.

Commit	Saves the staged changes in local repository history.
Push	Uploads local commits to the remote repository.
Fetch	Downloads remote history and updates remote-tracking references. [4]
Pull	Fetches remote changes and integrates them into the current branch. [5]

5. Remote Changes — Fetch & Pull

The remote README update is commit `3efa578`, “Document About feature completion.” After `git fetch origin`, local main stayed at `5cb5bce` while `origin/main` advanced to the new commit. Status showed main one commit behind; the pre-existing local README modification was still present [4].

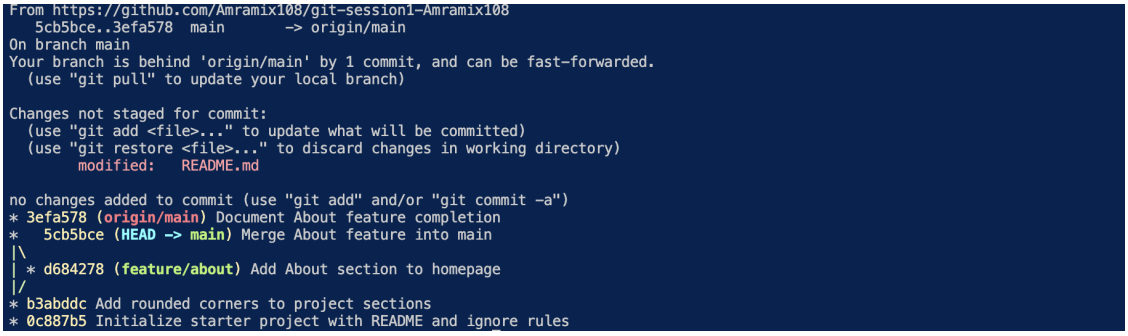


Figure 10. Fetch checkpoint: origin/main is ahead by one commit and the local README edit remains unstaged.

Before pulling, I saved the local README edit using `git stash push -m "Save README changes before pull" -- README.md`. This produced a clean working tree while main was still behind. I then ran `git pull origin main`, which fast-forwarded main to `3efa578` and added four lines to the README [5].

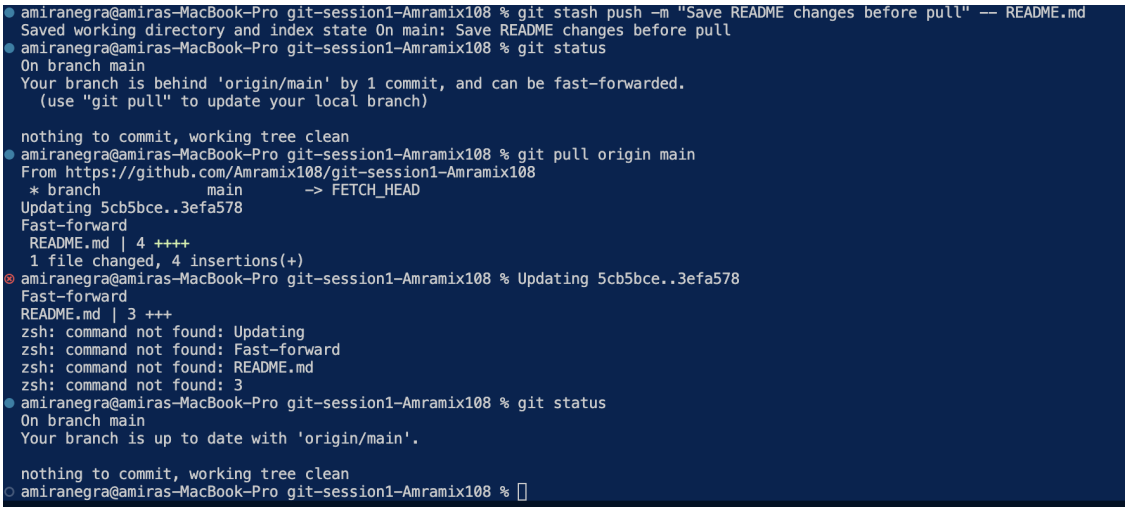


Figure 11. Stash, successful fast-forward pull, and final status showing main synchronized with origin/main.

The later `zsh: command not found` messages came from entering printed output as commands. The final Git status still confirms an up-to-date branch and a clean working tree.

Saved edits: a clean working tree after stashing does not mean the saved README edits have been committed. The screenshots do not show a later stash restoration; those edits need review [6].

7. Sources & Reflection

Sources

[1] Course slides: *Session 1: Git Basics & Your First GitHub Repository*.

[2] Provided *Assignment 1 and Report Template — Group D*.

[3] Git documentation — ignore patterns: https://git-scm.com/docs/gitignore

[4] Git documentation — fetch: https://git-scm.com/docs/git-fetch

[5] Git documentation — pull: https://git-scm.com/docs/git-pull

[6] Git documentation — stash: https://git-scm.com/docs/git-stash

Reflection

I did not find any concept especially difficult. The exercise helped me distinguish working-file changes, staged changes, saved stashes, and committed history, as well as the different effects of fetch and pull. If Git behaved unexpectedly, I would first inspect `git status`, `git remote -v`, and the recent commit graph before choosing the next command.